

Daml Development	2
Canton Transaction Basics	3

Daml Development

Objective: Get comfortable with building, testing, and deploying a Daml model.

- ☐ **Hands-on** - Write your first daml application (build and run with dpm sandbox)
- ☐ **Theory** - How can you inspect the contents of a DAR? What's inside?
- ☐ **Hands-on** - Test your first daml application with daml script
- ☐ **Theory** - what are signatories and observers?
- ☐ **Hands-on** - Implement and test the propose-accept pattern -
<https://docs.canton.network/appdev/modules/m2-multi-party-workflows#the-propose-accept-pattern>
- ☐ **Theory** - Consider also the delegation pattern:
<https://docs.canton.network/appdev/modules/m2-multi-party-workflows#delegation-patterns>. How does it compare to the propose-accept pattern? In which scenarios would you use each pattern?
- ☐ **Hands-on** - Launch localnet -
<https://docs.canton.network/appdev/modules/m5-localnet-development>
- ☐ **Hands-on** - Deploy your daml project to localnet
- ☐ **Hands-on** - Execute a daml transaction between two nodes - app user and app-provider (caution: documentation may be outdated):
<https://forum.canton.network/t/multi-participant-daml-scripts/7039>
 - src code:
<https://github.com/wallacekelly-da/daml-public-demos/tree/daml-script-participant-config>
 - Read the *API user rights* in the next section before attempting this. To generate jwt access_token for the participant config, use the same jwt-cli command in [entrypoint.sh](https://github.com/canton-network/entrypoint.sh)
- ☐ **Theory** - What issues arise when scaling a daml project to multiple nodes? How could those issues be mitigated when building a real-world Daml project?

Canton Transaction Basics

Focus: Learn about Canton Transaction Processing and How it Implements Privacy

- ☐ **Theory** - review our documentation on API user rights
 - <https://docs.digitalasset.com/build/3.4/sdlc-howtos/applications/secure/authorization.html#access-tokens-and-rights>
 - <https://docs.canton.network/appdev/deep-dives/authorization>
- ☐ **Theory** - learn about Canton transaction processing
 - <https://docs.daml.com/canton/architecture/overview.html>
 - <https://docs.canton.network/overview/learn/architecture>
 - <https://docs.canton.network/overview/learn/privacy-model>
 - <https://docs.canton.network/overview/learn/two-layer-consensus>
 - <https://docs.canton.network/overview/learn/ledger-model>
 - ☐ What is the role of each node in a Canton sync domain? What are the respective obligations of each node operator?
 - ☐ What are the confirming and observing nodes of a daml transaction? How do they relate signatories/observes and transaction authorizers/informees?
- ☐ **Theory** - how does Daml break down a transaction into nodes? What are transaction views? Why does the Daml engine break transactions into sub-transactions? - <https://docs.daml.com/concepts/ledger-model/ledger-structure.html#actions-and-transactions>
 - <https://docs.canton.network/overview/reference/ledger-model-detailed> (read from the section **Structure** to **Ledger**).
- ☐ **Hands on** - See if you can contrive a couple of daml choice definitions that generate different numbers of transaction views, but at the application level achieve the same goal. Reducing the number of views in a Daml transaction is a skill we call "view optimisation". Canton has some capability to optimise views for you, but in advanced cases optimising daml views can lead to material application performance improvements.
- ☐ **Theory** - learn how to infer the authorizers and informees of a transaction (and subtransaction)
 - <https://docs.canton.network/overview/reference/ledger-model-detailed> (read from the section **Privacy** to **Disclosure: When non-stakeholders use contracts**)
 - ☐ Authorizers - <https://docs.daml.com/concepts/ledger-model/ledger-integrity.html#authorization>
 - ☐ Informees - <https://docs.daml.com/concepts/local-ledger.html#local-ledgers>
 - ☐ Bottom line - what is the relationship between authorizers and informees?

Focus: Off-ledger automations and trigger patterns

- ☐ **Theory** Review the openAPI spec. How could one perform the following tasks?
 - <https://docs.digitalasset.com/build/3.4/reference/json-api/openapi.html>

- <https://docs.canton.network/sdks-tools/api-reference/json-api>

- <https://docs.canton.network/reference/json-api-reference>

- ☐ Upload a dar
- ☐ List dars
- ☐ Allocate a party
- ☐ Allocate an external party
- ☐ Grant rights from a user to a party
- ☐ Create a contract
- ☐ Exercise a choice
- ☐ Check on the status of a submitted command by command ID
- ☐ Explore a transaction by update ID
- ☐ Explore a transaction by offset
- ☐ List active contracts
- ☐ Check a contract's contents and type by contract ID
- ☐ Check if a contract is archived by ID
- ☐ Get the participant node's latest offset, and last pruned offset
- ☐ **Theory** review the gRPC ledger API. This is actually our more hardened API offering, while the JSON API is newer and still has UX teething issues.
<https://docs.digitalasset.com/build/3.4/explanations/ledger-api-services.html>
<https://docs.canton.network/sdks-tools/api-reference/ledger-api-services>
- ☐ **Hands-on** - write an off-ledger automation that observes contract proposals and auto-accepts them.
- ☐ **Theory** - Investigate the ledger clients we offer. "Theory" because this part of our product offering is in a state of volatility.
 - ☐ OpenAPI generator
 - ☐ gRPC ledger api
 - ☐ Our Java ledger api wrappers
 - ☐ Java codegen
 - ☐ JS/typescript codegen
 - ☐ Transcode
- ☐ **Hands-on** - configure and deploy PQS against localnet, explore PQS
 - <https://docs.canton.network/appdev/modules/m4-query-with-pqs>
 - <https://docs.canton.network/sdks-tools/development-tools/pqs>
 - <https://docs.canton.network/appdev/reference/pqs-sql-reference>
- ☐ **Theory** - what value does PQS offer over using the ledger API directly?
<https://archived.docs.digitalasset.com/build/3.4/component-howtos/pqs/references/architecture.html>
- ☐ **Theory** - what APIs does PQS provide for "freezing" the ledger offset during queries? What is the use case/benefit of such APIs?
- ☐ **Hands-on** - integrate PQS into your application

- ☐ **Theory** - research error handling. Which errors are likely to be transient errors? Which ones are likely to depend on the application design?
- <https://docs.canton.network/appdev/reference/error-codes>

Dec 4, 2025

Focus - Pruning

- ☐ **Hands-on** - Trigger an ad-hoc prune of a participant node
- ☐ **Hands-on** - Schedule pruning for a participant node
- ☐ **Theory** - What data is lost when pruning happens? What should a user do if they want to retain that data?
- ☐ **Theory** - What changes need to be made to a ledger client application to make them pruning-aware?
 - ☐ How should they be architected to do a state rebuild from the ledger history?
 - ☐ How should they be architected to survive a case where they've been offline for longer than the pruning window?

Focus - Observability

- ☐ **Hands-on** - set up Canton's Inav plugin and experiment with searching through the logs. See if you can perform the following common tasks:
 - ☐ Filter logs by trace ID
 - ☐ Filter logs by event type
 - ☐ Navigate the logs by timestamp
 - ☐ Show the timeline margin
 - ☐ Enable mouse controls
- ☐ **Theory** - how does one set the trace ID when performing a ledger submission?
- ☐ **Hands-on** - Set up a canton participant node whose metrics are published and shown in a Grafana dashboard
- ☐ **Theory** - Could you use these metrics to determine transaction processing throughput and latency?
- ☐ **Theory** - Could you use these metrics to determine ACS size and distribution?

Focus - Validator Nodes

- ☐ **Hands-On** - launch a devnet validator node with kubernetes
https://docs.dev.sync.global/validator_operator/validator_helm.html
- ☐ **Hands-On** - deploy a validator node with OAuth integration (eg keycloak)

Focus - External Party Hosting

- ☐ **Hands-On** - allocate an external party on localnet
- ☐ **Theory** - what rights does a ledger API user need to submit a command on behalf of an external party?

- ☐ **Hands-On** - submit an externally-signed transaction
<https://docs.digitalasset.com/integrate/devnet/preparing-and-signing-transactions/index.html>
- ☐ **Hands-On** - allocate an external party hosted on multiple nodes.
- ☐

Jan 6, 2026

Focus - Canton Coin and Token Standard Transfers

- ☐ **Hands-On** - Perform a Canton Coin transfer with the wallet UI on localnet
- ☐ **Hands-On** - Set up transfer preapproval on localnet. How does transfer preapproval change the transfer workflow?
- ☐ **Theory** - What ledger commands establish the transfer preapproval? What automations are involved? What Daml design pattern is used here?
- ☐ **Theory** - the token standard APIs for metadata, holdings, and transfers. What API call(s) are required to perform a token standard transfer of Canton Coin?
- ☐ **Theory** - More generally, what role do the token standard APIs play in token standard transfer? Why can't everything just be done through the ledger API?
- ☐ **Hands-On** - perform a token-standard transfer of Canton Coin using the token standard and ledger APIs.
- ☐ **Theory** - How do the API interactions differ between a token standard transfer with transfer preapproval, and one without? How can an API user expect in advance of ledger submission that a transfer preapproval will be used?
- ☐ **Theory** - how can token-standard transfers be used to split and merge Canton Coin contracts?
- ☐ **Theory** - how can one perform a bulk transfer of Canton Coin (for example, for air-drops)?
- ☐ **Theory** - what advice would you give to an application developer looking to manage the number of Canton Coin contracts that they have?

Focus - Allocations

- ☐ **Theory** - Finish reviewing the token standard APIs. What is the relationship between allocations, allocation requests, and allocation instructions? Which one is most likely to come first?
- ☐ **Theory** - What part of the token standard API executes the allocation transfer?
- ☐ **Hands-On** - Set up transfer preapproval on localnet. How does transfer preapproval change the transfer workflow?
- ☐ **Theory** - What ledger commands establish the transfer preapproval? What automations are involved? What Daml design pattern is used here?
- ☐ **Theory** - the token standard APIs for metadata, holdings, and transfers. What API call(s) are required to perform a token standard transfer of Canton Coin?

- ☐ **Theory** - More generally, what role do the token standard APIs play in token standard transfer? Why can't everything just be done through the ledger API?
- ☐ **Hands-On** - perform a token-standard transfer of Canton Coin using the token standard and ledger APIs.
- ☐ **Theory** - How do the API interactions differ between a token standard transfer with transfer preapproval, and one without? How can an API user expect in advance of ledger submission that a transfer preapproval will be used?
- ☐ **Theory** - how can token-standard transfers be used to split and merge Canton Coin contracts?
- ☐ **Theory** - how can one perform a bulk transfer of Canton Coin (for example, for air-drops)?

Focus - Tokenomics

- ☐ **Theory** - Read the Canton Coin tokenomics whitepaper
- ☐ **Theory** - How does validator auto top-up work? What is the relationship between traffic and Canton Coin? How can one "buy more traffic" for another validator?
- ☐ **Hands-on** - Configure the traffic top-up rate for a validator node
- ☐ **Hands-on** - Research app activity markers. Add app activity markers to your Daml project
- ☐ **Theory** - What are the rules/guidelines for app activity markers? How do app activity markers lead to more Canton Coin?
- ☐ **Theory** - What are the benefits to the network of using the transfer preapproval? How do we reward use of transfer preapproval?

Next

Focus - Contract Upgrades

- MUT
- SCU

Focus - Utilities

- MUT
- SCU